

Libraries

- files of code that other people wrote so that we can use them, or a library's code that was written by me so that they can be written in other programs. Most of these take in the form of modules.

The "random" module

random.choice(seq)

- "seq" stands for "sequence" and generally means a list or a list-like object.

"from" allows users to specify which function from a module to import

ex. `from random import choice` ← imports the "choice" function from the random module

`coin = choice("heads", "tails")`



"choice" function is now in the namespace of this file

the "choice" function in the random module chooses any item/value in a list at random (50% heads, 50% tails probability)

Although it may be convenient to simply call the module's function instead of specifying the module it came from then calling it, this could also be detrimental because now the function name is in our namespace.

random.randint(a, b)

- get a random int between a and b inclusive, and return it

random.shuffle(x)

- takes in a list of values and shuffles them up, randomizing the list order
- does not return the list; shuffles the list in place

ex. import random

```
num = random.randint(1, 10)
print(num)
```

```
Cards = ["pee", "poo", "puu"]
random.shuffle(Cards)
print(Cards)
```

not stored
in a variable

{random number} 4
{random shuffle} ['poo', 'pee', 'puu']

The "Statistics" module

Statistics.mean(seq)

- takes in a list of values (int) and returns the average

Command-line arguments

- allows the user to provide arguments that is inputted into the program
Just when it is being executed at the command line

The "Sys" module

Contains functions that are specific to the system itself and the commands that are being typed by the user

* sys.argv

- "argv" stands for "argument vector" and describes the list of all words the user typed in the prompt before they hit enter
- This is one of the functions that utilize Command-line arguments.

ex.

```
name.py
import sys
```

```
print("Hello. This is", sys.argv[1])
```

terminal

```
>>> python name.py Poob
Hello. This is Poob
```

sys.argv[0] is the name of the file, "name.py"

sys.argv[1] is the CLA, "Poob"

* `sys.exit`

- exits the program when called
- Can also take in arguments (str) to be called right when exiting

ex.

```
name.py
import sys
if len(sys.argv) < 2:
    sys.exit("perms")
elif len(sys.argv) > 2:
    sys.exit("poornis")
print("yo,", sys.argv[1])
```

```
terminal
>>> python name.py
perms
>>> python name.py poo poo
poornis
>>> python name.py shee
yo, shee
```

`sys.argv` can be used in a for loop (it is a list).

ex.

```
name.py
import sys
for arg in sys.argv:
    print("yo,", arg)
```

```
terminal
>>> python name.py poo pee po
yo, name.py
yo, poo
yo, pee
yo, po
```

* shee here with `[1:]`
to avoid including "name.py"

Packages

- third party libraries, which are modules implemented in a folder that can be downloaded to gain access to more functionalities

* more about slicing in GTPy workspace

PyPI website

- the Python Package Index (pypi.org)
- allows the user to download and install many different packages

CowSay (example)

- a package in python that allows the user to have a Cow say something

How to get the package into the system:

method 1: download the zip file, unzip, put in the proper directory

method 2: pip

Lots of programming languages have a package manager. Python has a program called "pip" that allows the user to install packages by just running a command.

ex. installing CowSay

terminal

```
>>> pip install CowSay
```

↑
pip program
is called

↑
what to do

↑
package name

ex. Cont. Using CowSay

Say.py

```
import CowSay  
import sys
```

```
if len(sys.argv) == 2:
```

```
    CowSay.cow("hello, " + sys.argv[1])
```

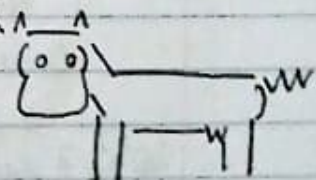
"cow" function can only pass in 1 string

There is also the .trex() function!

terminal

```
>>> python Say.py Poob
```

```
hello, Poob
```



ASCII art!

We use Command-line arguments because of productivity. It allows the user to get faster by getting in the habit of knowing how to configure software without having to manually answer questions.

APIs

- Application Programming Interface that refers to 3rd party services

requests library allows you to make web requests, using Python as though it were a browser.

Some APIs are formatted in JSON (JavaScript Object Notation), which is a language agnostic format for exchanging data between computers.

(language agnostic, meaning it is not strictly applied to javascript). Essentially, it is just a bunch of text that has been formatted into dicts and lists, stored in a file, and the file itself is an API.

requests.get

- a function inside the requests package that will get the response from a server

ex. getting data from an API

```
import requests
```

```
response = requests.get("____URL____")  
print(response.json())
```

uses the "get" function to get the response from a server in the specified URL.

The response is stored in a variable named "response".

Convert the response into a json file

Python also has a library called "JSON" that manipulates a JSON file data to pretty print it, and format it in a way that is more legible to the user.

Json.dumps (file / data)

- also called dump String, accepts a response and pretty prints the file.
- this helps the user read the data / file more clearly and see the structure of the file.

ex. response = requests.get ("_____URL_____")

o = response.json()

for result in o["look1"]:

print (result["look2"])

(assume "look1" and "look2"
are keys in a response of
a json file)

Making a module

- basically, making a module is making a file with some useable code.

If a module itself calls a function within that file and another file imports that module, the function that has been called within the module will also be called.

ex. module.py
def main():
 hello("world")

def hello(name):
 print(f"hello, {name}")

main()

main gets called
inside the module

go.py
from module import hello
import sys

hello(sys.argv[1])

terminal
>>> python go.py Poob
hello, world
hello, Poob

main is called

To prevent the previous error:

apply this in the module:

```
if __name__ == "__main__":  
    main()
```

In Python, `--name--` is a special variable whose value is automatically set to "main" when you run a file from the command line.

Ex. running `module.py` with `--name--`

```
>>> python module.py  
hello, world
```

running `yo.py` with `--name--`

```
>>> python yo.py Poob  
hello, Poob
```